

Propuesta

Chatbot para Técnicos (con vision de computadora)

Felix Omar Dominguez Inestroza - 12141043
Samuel Andres Rocha Castro - 21941333



Índice

Planteamiento del problema

Descripción de la Solución

Objetivos

Modelos de Aprendizaje de Maquina

Arquitectura del sistema

Revisión Literaria

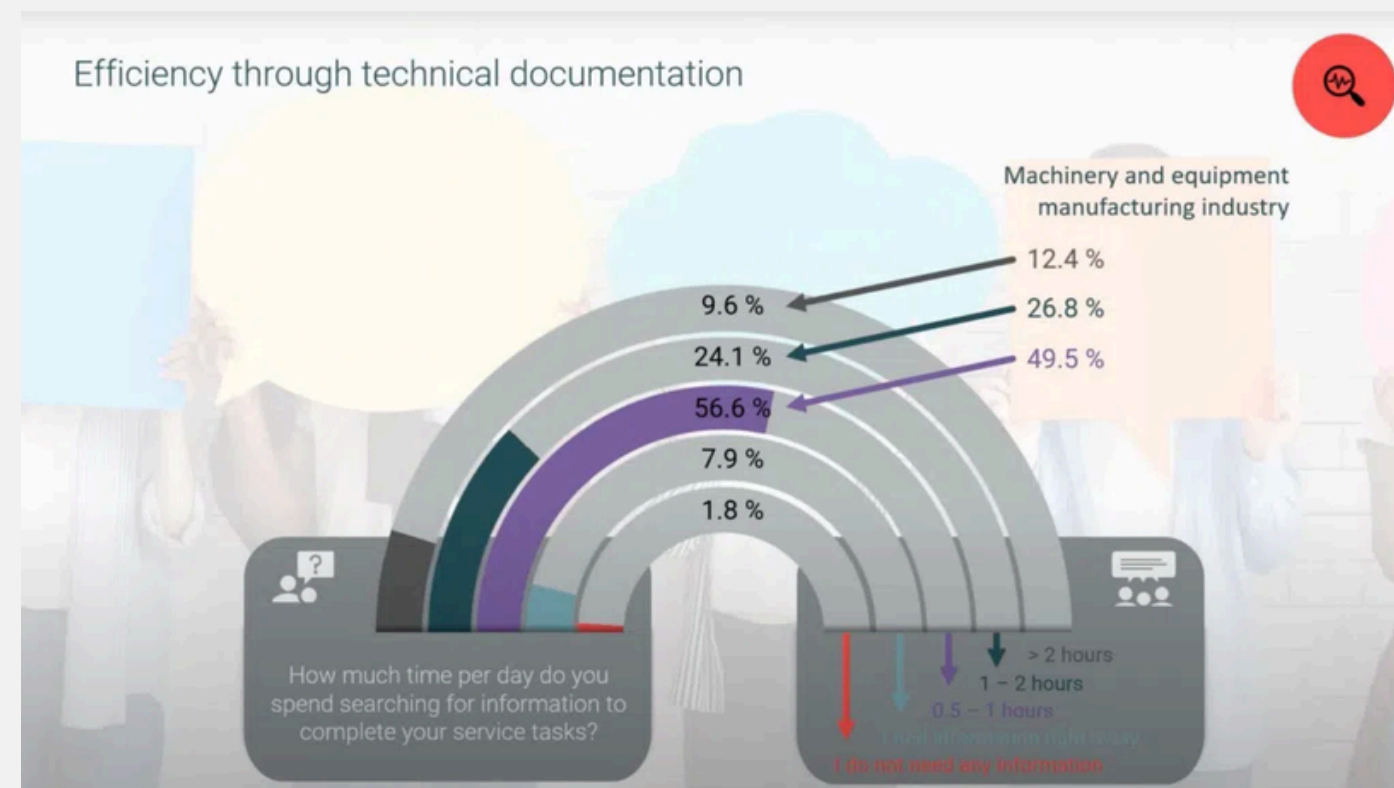
Implementación

Resultados

Bibliografía

Planteamiento del Problema

El 80% de los técnicos de campos se suelen tardar entre 30 minutos a 2 horas en solo buscar la información necesaria para hacer su trabajo, además recalcan el tiempo que se pierde en buscar dentro de la documentación misma. También el 95% de los casos los técnicos tuvieron que agendar una segunda visita a su cliente simplemente porque no contaban con la información necesaria. Por esto podemos ver los diferentes problemas a los que se afrontan los técnicos los cuales generan perdida de tiempo, errores de diagnostico y retrasos a la atención al cliente.
(Muffat, 2023)



Objetivo General

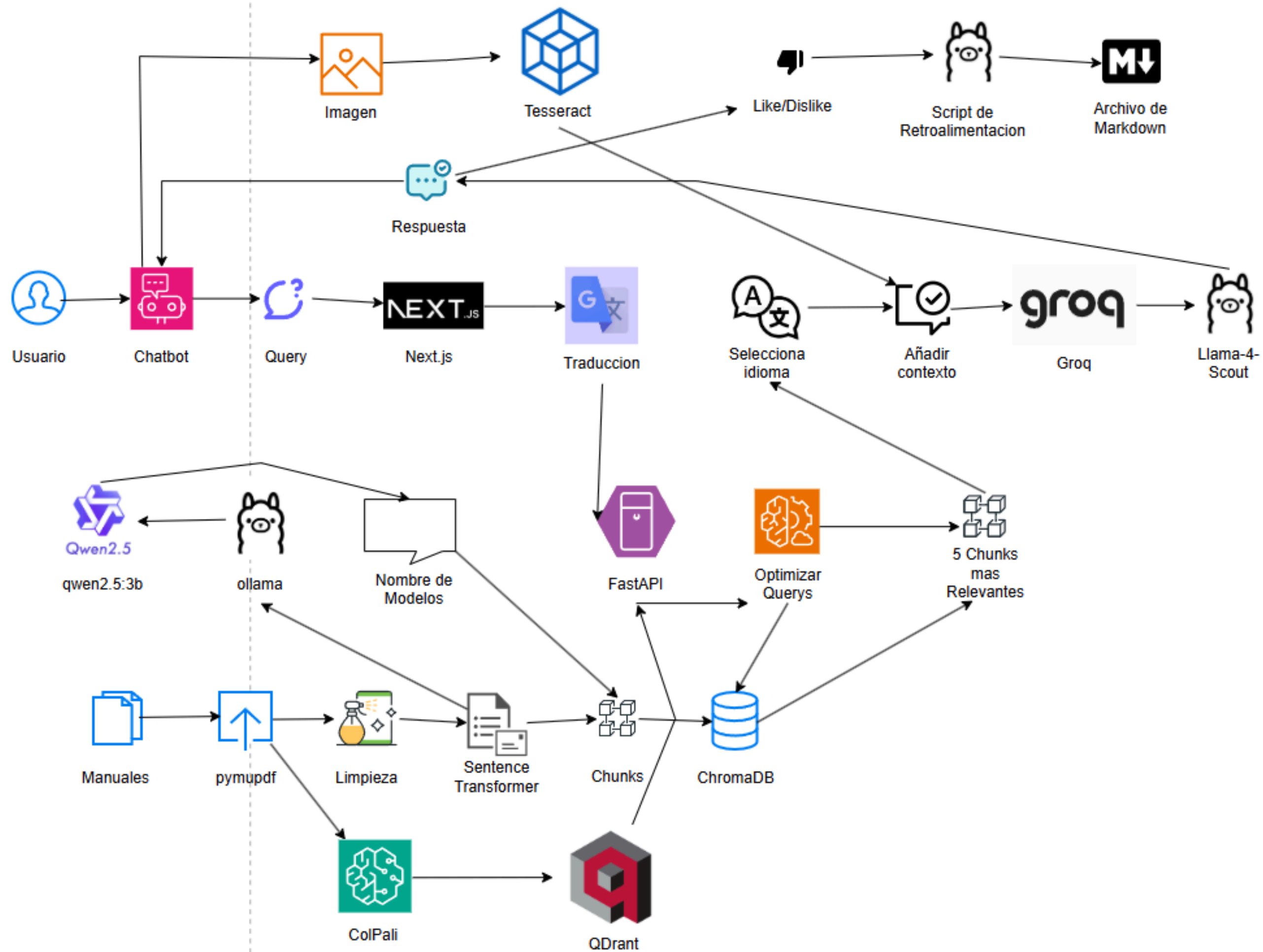
Desarrollar un sistema de RAG orientado a técnicos de campo en el área de dispositivos de refrigeración, integrado en un chatbot para facilitar la consulta rápida y precisa de manuales técnicos a través de interacciones en lenguaje natural y visión por computadora en forma de imágenes.

Objetivos Específicos

- Rediseñar el modelo RAG que ya tenemos para que funcione para almacenar correctamente información visual como imágenes.
- Implementar visión de computadora para que por medio de fotos pueda ser capaz de identificar los dispositivos en los cuales se van a trabajar, específicamente la de su etiqueta técnica.
- Brindar una forma de dar retroalimentación a los desarrolladores

Descripción de la Solucion

Desarrollar un chatbot utilizando un sistema de RAG el cual sea capaz de brindar respuestas a preguntas sobre dispositivos electrodomesticos, enfocados en los dispositivos de refrigeracion, las cuales se podrian encontrar de sus respectivos manual, brindando ambas respuestas textuales ademas de brindar graficas, dibujos y diagramas como respuestas. Ademas, incorporar un sistema de reconocimiento por imagenes de los dispositivos de los cuales tienen consultas.



Arquitectura



Modelos de ML



Etapa de Indexado

- ColPali para procesamiento de documentos con imágenes.
- Colpali-Engine para la implementación del modelo

Etapa de Generación

- Llama 4 Scout con soporte para imágenes.
- Implementación del modelo con la API de Groq.

OCR

- Tesseract
- TessTrain para entrenar Tesseract

ColPali

Modelos de ML



ColSmol

- Usa SmolVLM como VLM
- Tiene modelos de 500M y 250M de parametros

ColPali

- Usa PaliGemma como VLM
- Tiene 3B de parametros

ColQwen

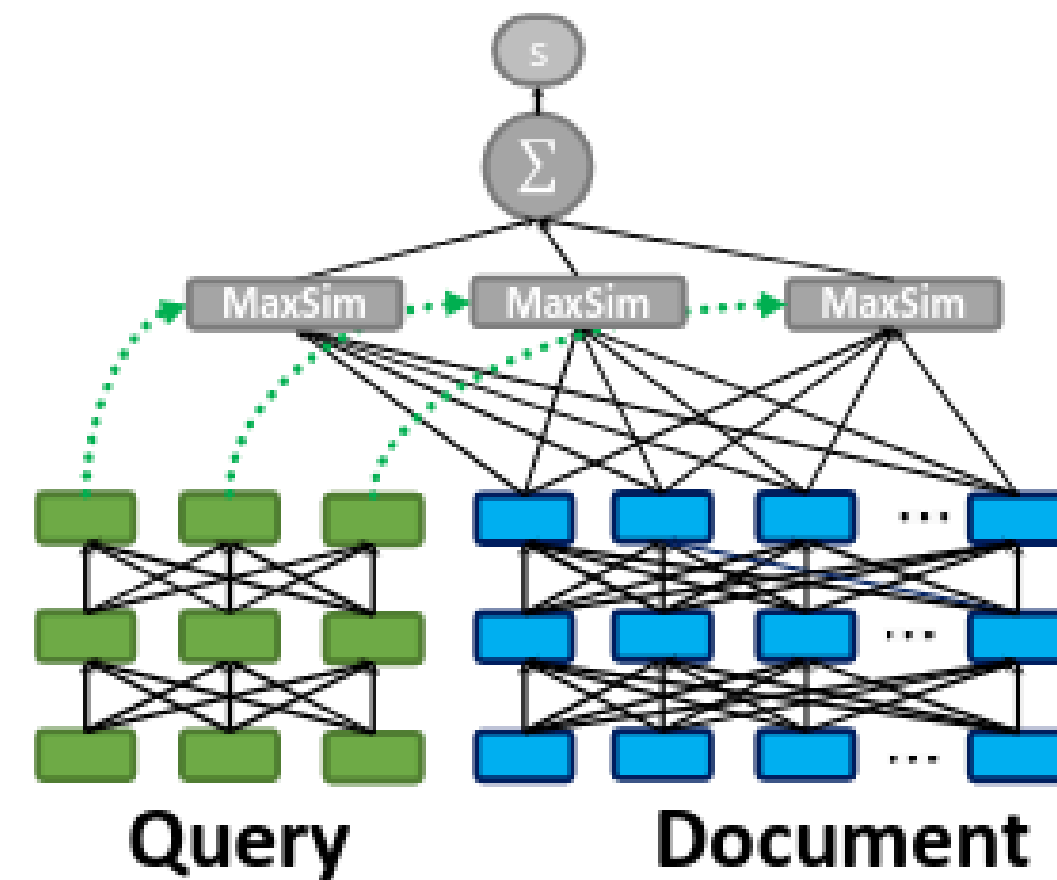
- Utiliza Qwen 2 o 2.5 como VLM.
- Usa 2B o 3B de parametros segun el modelo

ColBERT

Modelo de recuperación y ranking basado en el Language Model BERT que introduce vectores de multiples dimensiones e interacción tardía entre un par query, documento.

Arquitectura

- Codificador de Query
- Codificador de Documentos
- Late Interaction (LI)

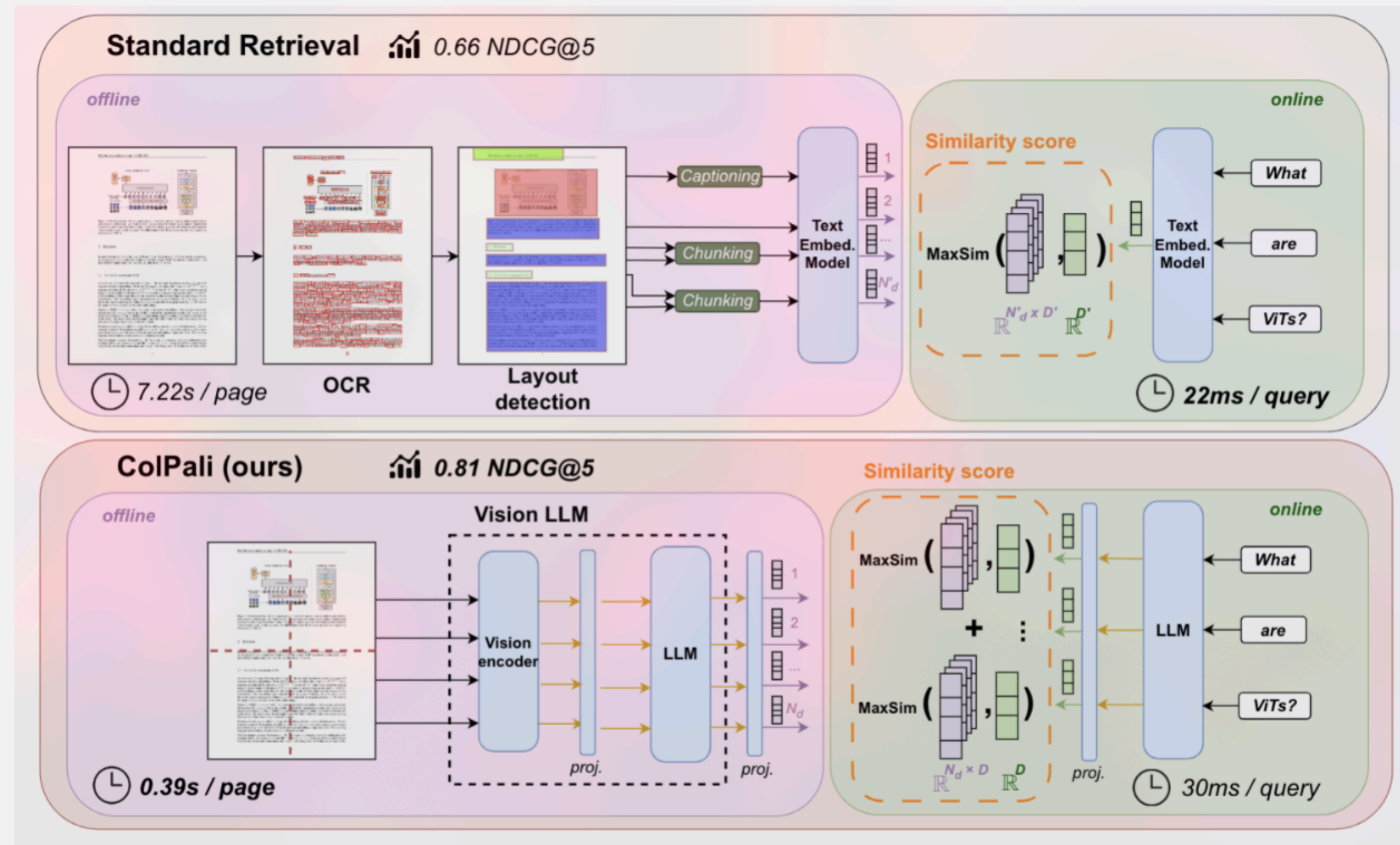


(d) Late Interaction
(i.e., the proposed ColBERT)

ColPali

El modelo ColPali, desarrollado por Illuin Technology, basado en ColBERT permite utilizar la misma arquitectura reemplazando el Language Model (LM) por un Vision Language Model (VLM) permitiendo procesar cada página del documento como una imagen.

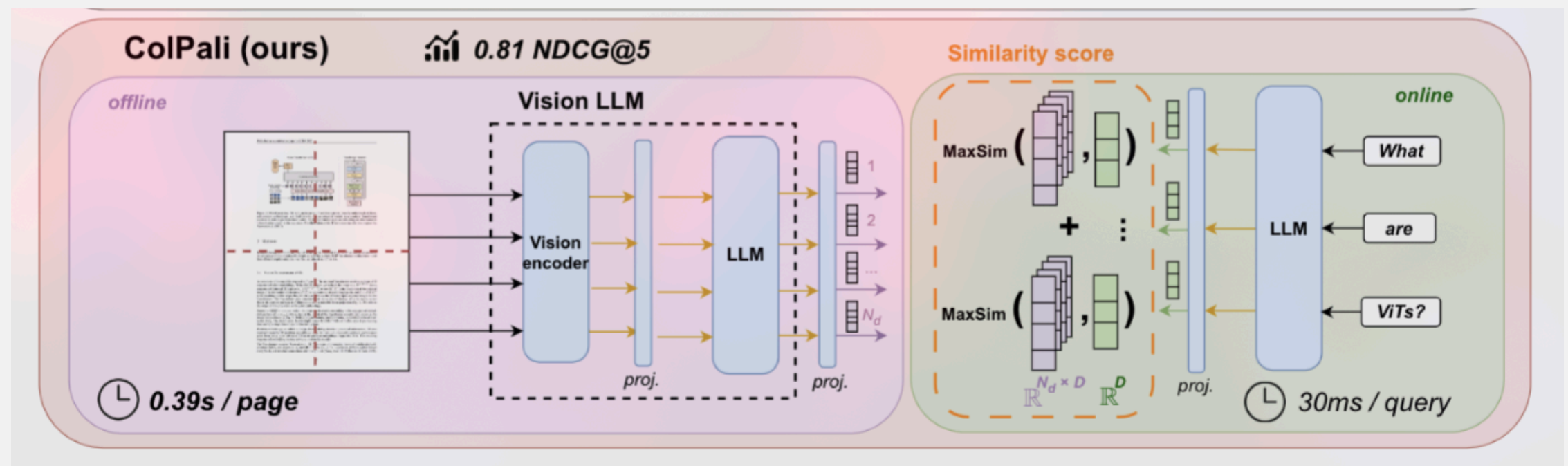
En lugar de extraer texto, el VLM divide cada página en patches visuales, generando un embedding por región.



ColPali

Arquitectura

1. Vision Language Model (VLM)
2. Late Interaction
3. Contrastive Loss



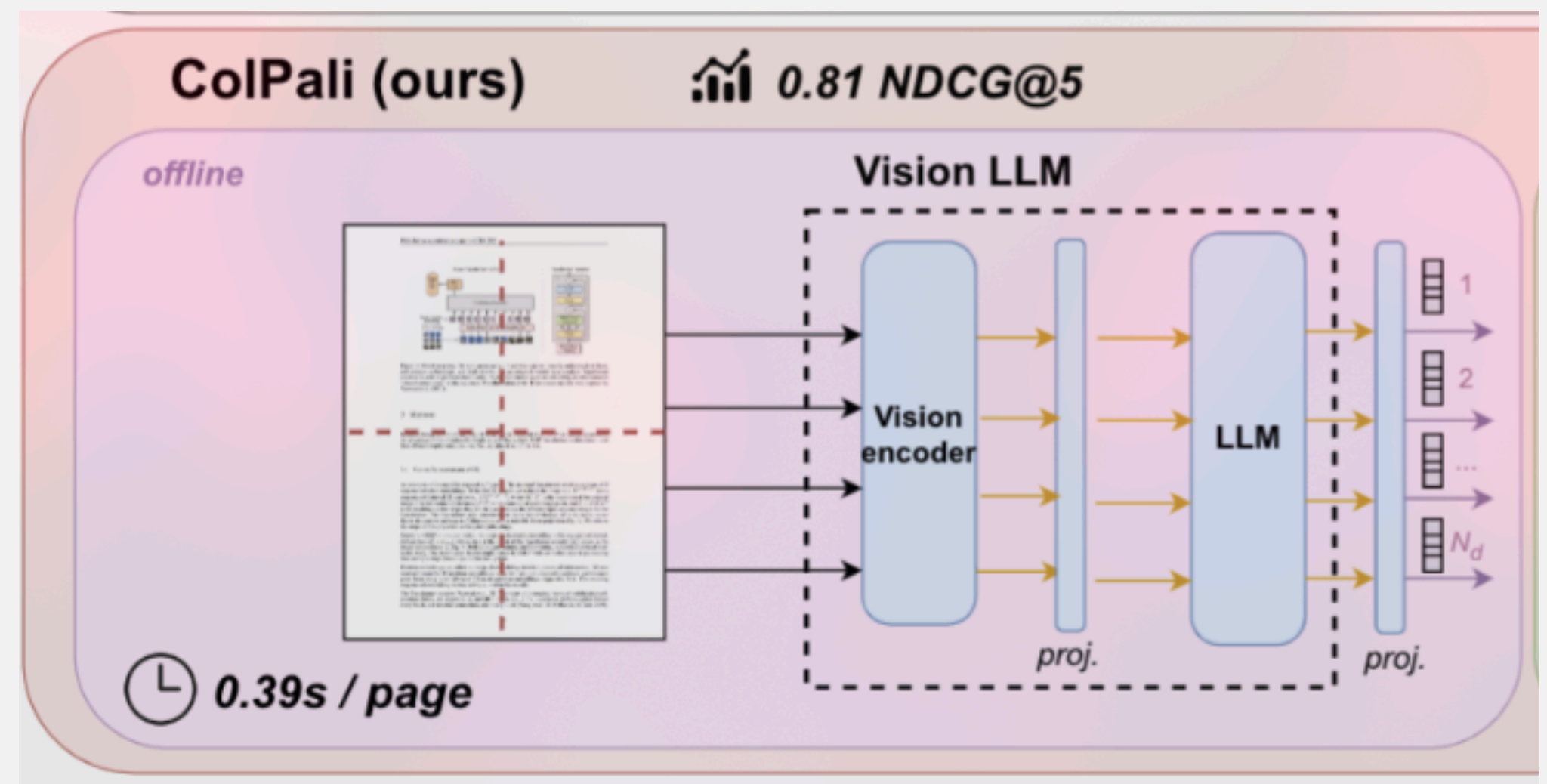
ColPali

VLM

El Vision Language Model (VLM) actúa como un encoder que genera embeddings para los documentos y las queries.

Para documentos:

1. Recibe la imagen de la página. SigLIP divide la imagen en patches.
2. Convierte cada patch en un vector, representación visual de esa región del documento.
3. Output: N embeddings finales. Un vector de embeddings por patch.

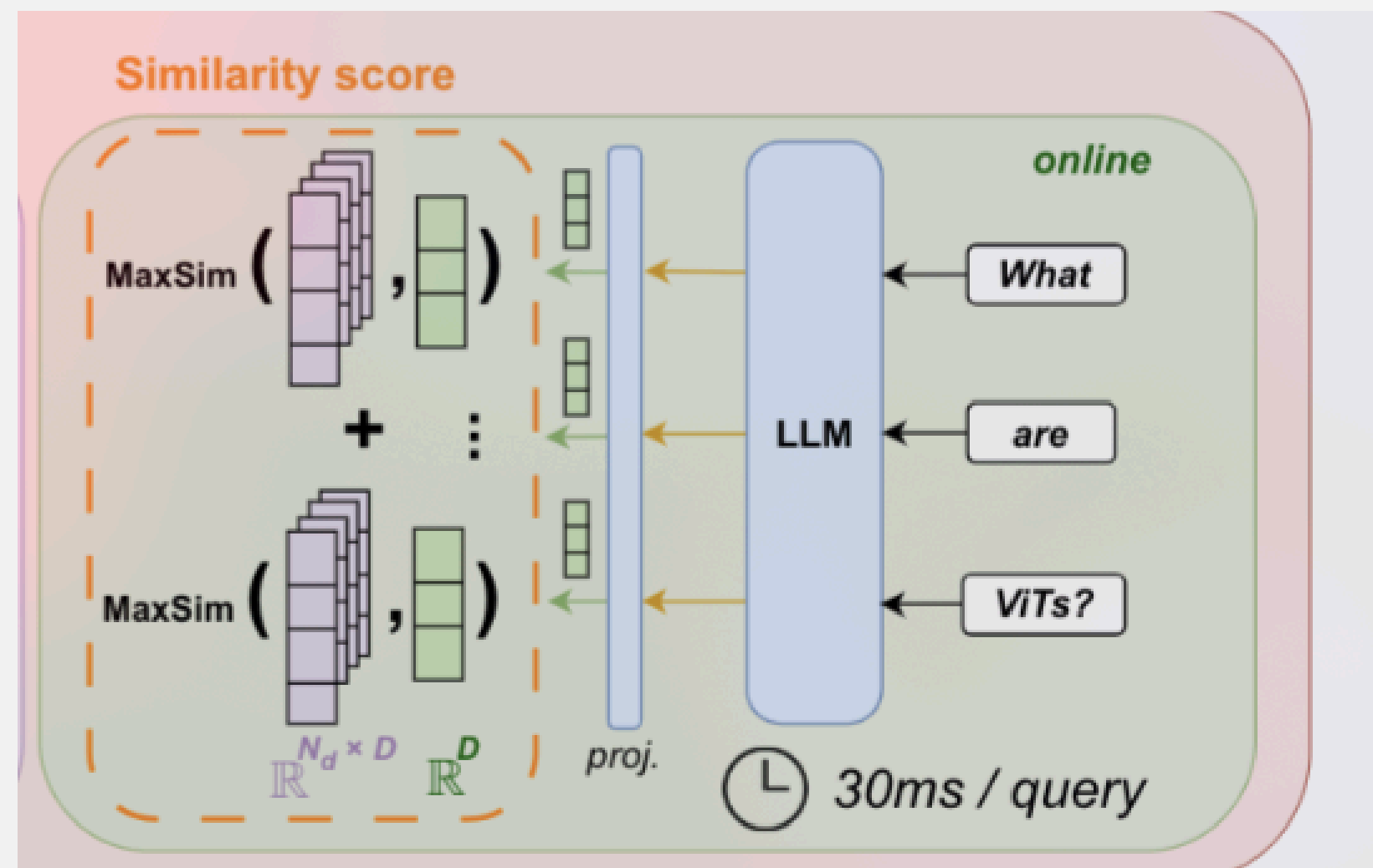


ColPali

VLM

Para las querys:

1. Recibe el texto de la query. Se divide el texto en tokens.
2. El VLM procesa los tokens como un LM normal.
3. Output: M embeddings, uno por token.



ColPali

Late Interaction y MaxSim

Late Interaction se utiliza para calcular el puntaje de relevancia de un documento **d** a una query **q** mediante la función MaxSim.

Donde:

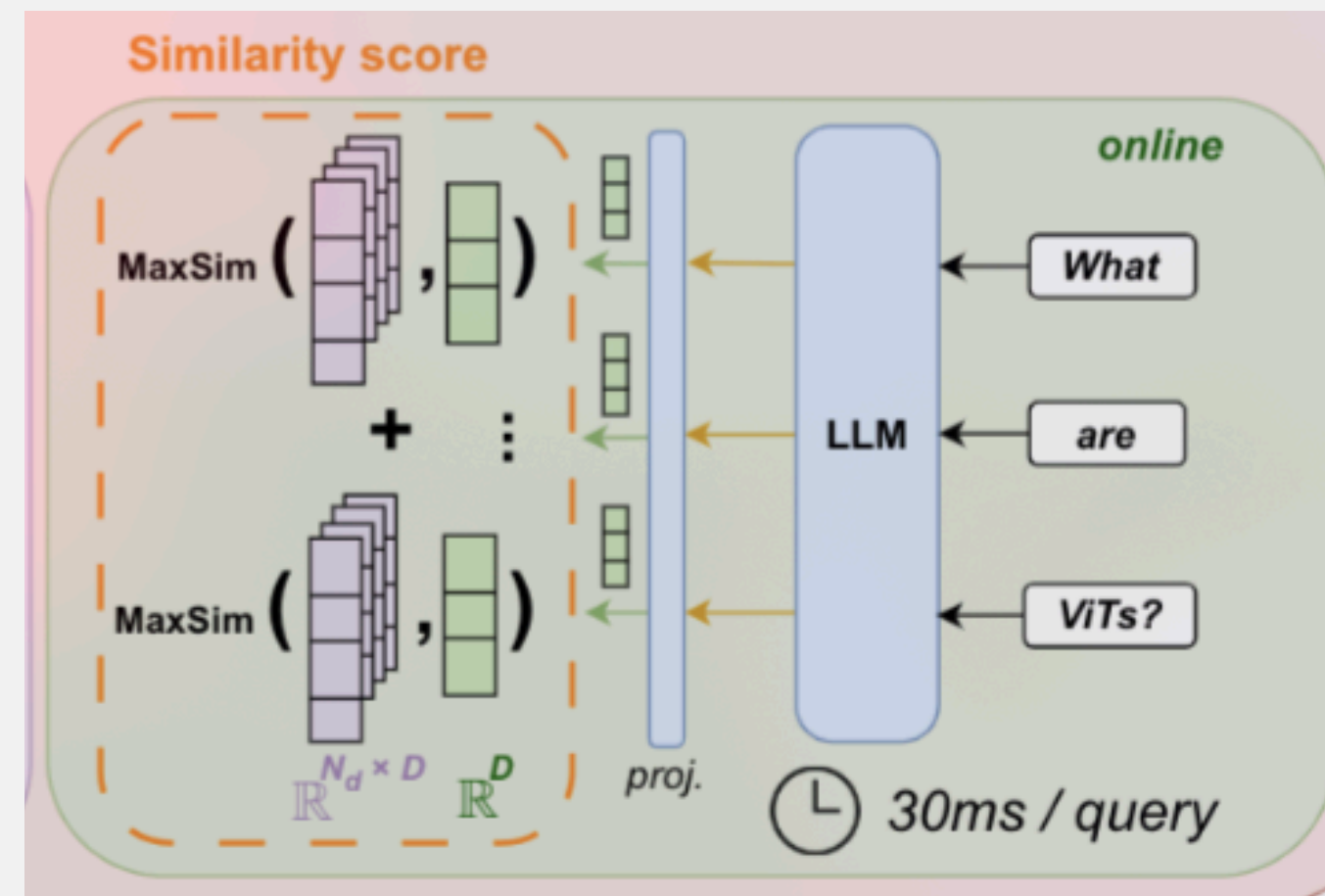
E_q Embeddings de la query **q**

E_d Embeddings del documento **d**

$|E_q|$ Cantidad de tokens en la query

$|E_d|$ Cantidad de patches en el documento

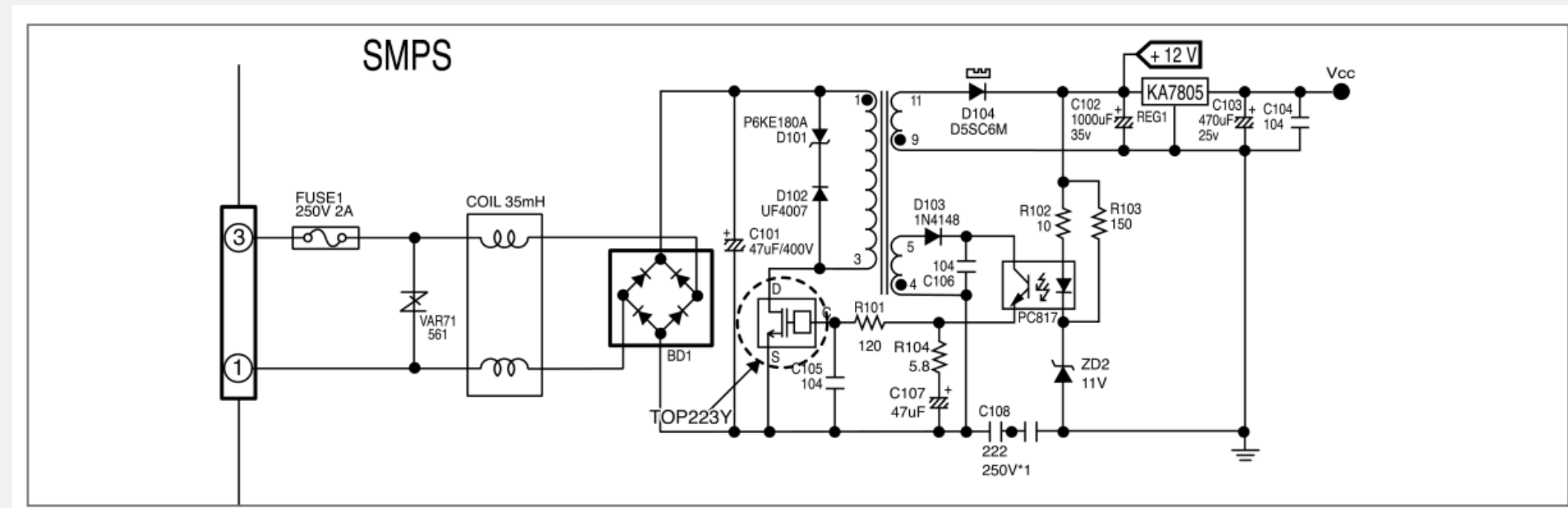
$$S_{q,d} := \sum_{i \in [|E_q|]} \max_{j \in [|E_d|]} E_{q_i} \cdot E_{d_j}^T$$



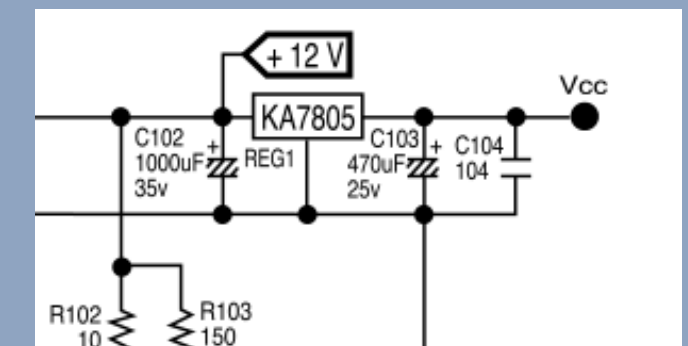
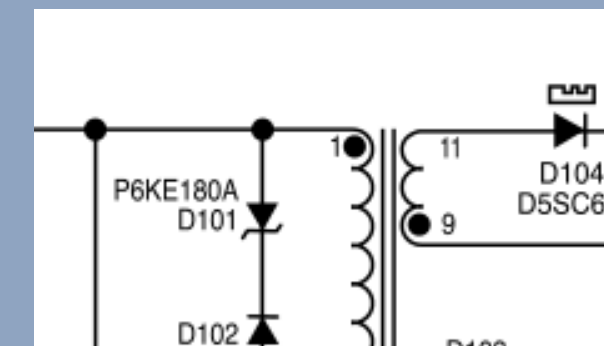
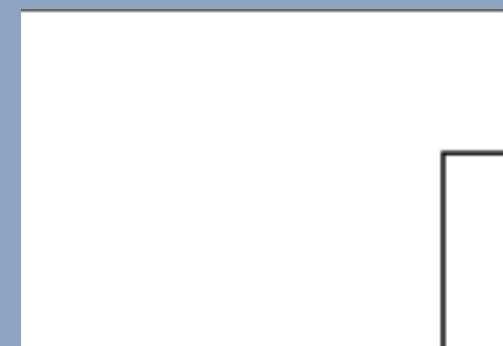
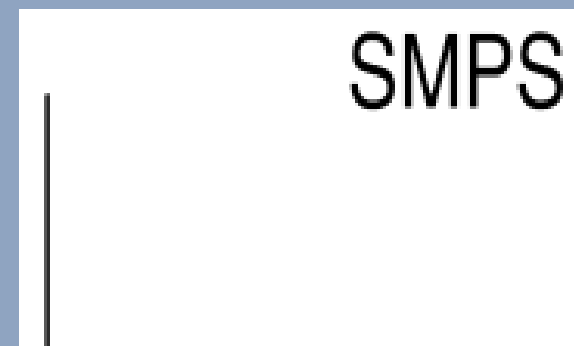
ColPali

Layouts

La recuperación basada en imágenes permite mantener la estructura de los layouts del documento a diferencia del OCR que colapsa la página a texto plano.



Colpali



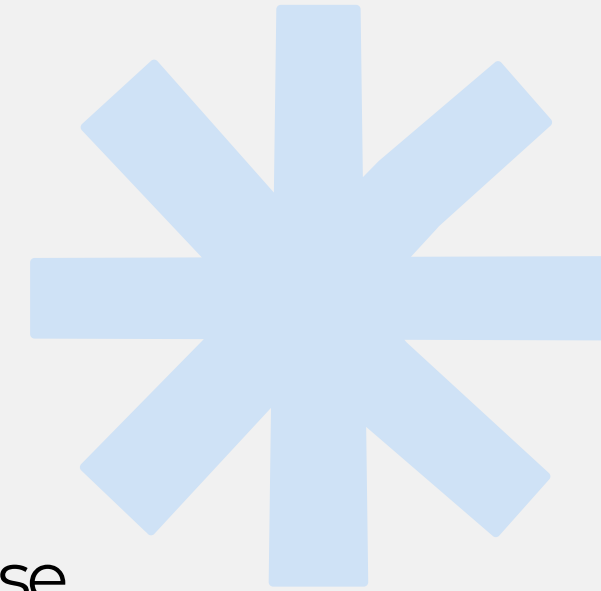
OCR

SMPS P6KE180A D101 D104 d102 d5sc6m c102 1000uF 35v ...

Tesseract



Tesseract OCR



Tesseract es un motor de OCR utilizado para extraer el texto que se encuentra en una imagen.

Razón de Uso:

- Código Abierto
- Muy ligero
- Enfocado a OCR nada más

Aplicación:

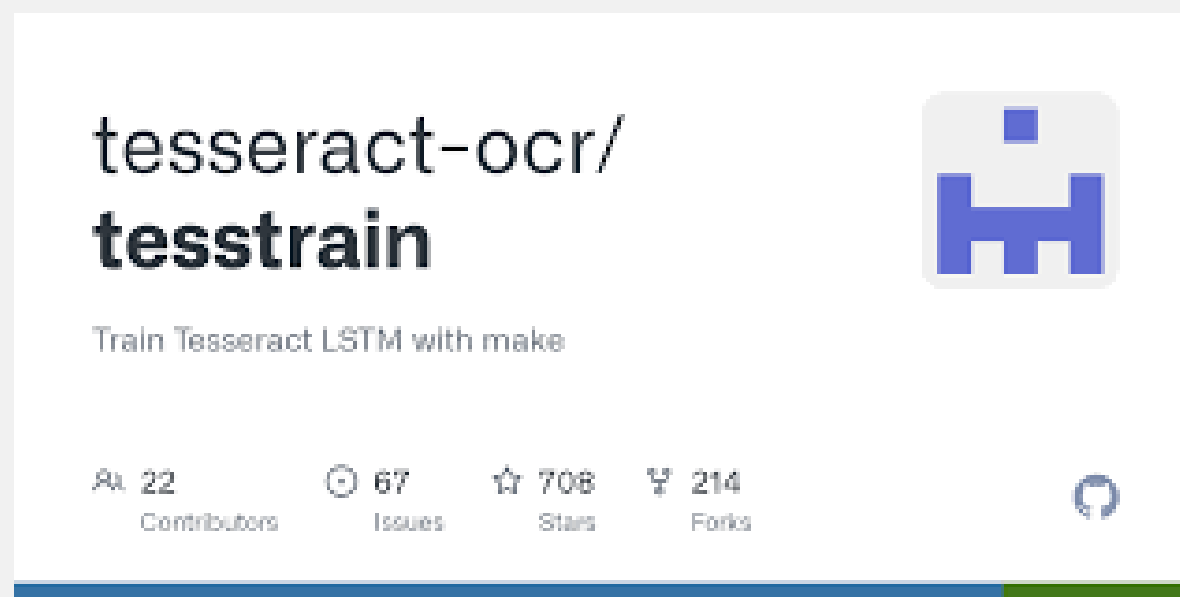
Para el desarrollo de este proyecto se utilizó Tesseract para extraer el texto de las etiquetas técnicas de los dispositivos a los que vamos a estar dando respuestas basadas en los manuales, para que identifique elementos clave de estos y los utilice para buscar en el RAG.

TessTrain

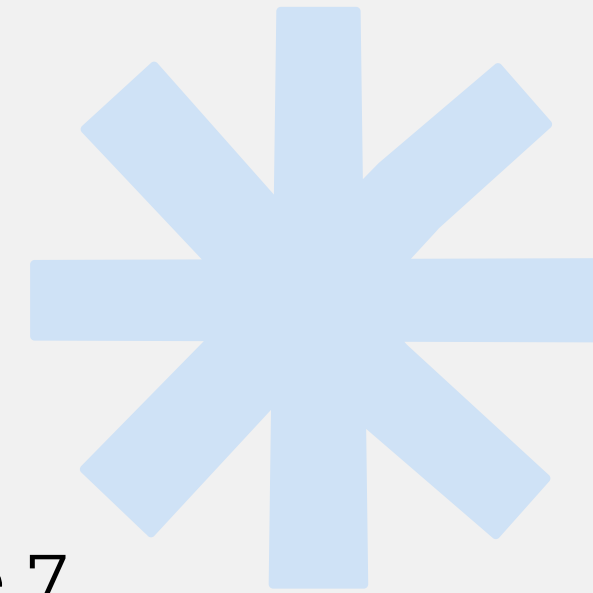
Para aumentar las capacidades del modelo original de Tesseract se utilizó TessTrain.

TessTrain es un proyecto de GitHub que se utiliza para entrenar al modelo base de Tesseract utilizando un conjunto de datos con pares, donde los pares serán archivos con el mismo nombre, pero uno siendo un archivo de imagen y el otro siendo un archivo .gt.txt. Estos se utilizan para brindarle la imagen al modelo y la respuesta correcta del texto que debería extraer de esta.

Para el modelo también se utilizó el modelo de eng.traineddata, el cual es una serie de modelos creados en el mismo repositorio de GitHub para entrenar a base de lenguajes.



Entrenamiento de Tesseract



Para entrenar Tesseract se utilizó un conjunto de pares de líneas de 7 placas técnicas que en total nos dieron 197 pares para entrenar.

Todas estas se realizaron a mano con un script para cortar los trozos de las placas y luego escribir en .gt.txt la respuesta correcta, debido a que no existía ningún dataset existente para la realización de esto.

Para el entrenamiento de Tesseract no se puede mandar toda la placa, ya que funciona solamente línea por línea; por ello es que se tuvo que hacer el recorte línea por línea.

Se utilizó el modelo en inglés debido a que la gran mayoría de estas estaban en inglés y este ayuda más con formatos internacionales.

Entrenamiento de Tesseract



ELECTRICAL RATING | NOMINAL VOLTS: 208/230

Ground Truth: ELECTRICAL RATING NOMINAL VOLTS: 208/230

ELECTRICAL RATING | NOMINAL VOLTS: 208/230

Respuesta de Tesseract: ELECTRICAL RATING NOMINAL VOLTS: 208/230

Retroalimentación



Para aplicar la retroalimentación, primero intentamos aplicar un sistema de configuración que, aunque no es verdadera retroalimentación, nos da un mayor control sobre el agente.

Hay dos opciones: Tipo de Respuesta y Tono de Voz.

Tipo de Respuesta: Solo RAG, RAG + Extra y RAG como referencia.

Tono de Voz: Casual, Normal, Formal.

Estos en sí solo eran cambios al prompt que se enviaba a Groq en tiempo real, mediante variables en el prompt.

Retroalimentacion

El verdadero metodo de retroalimentacion, utilizamos el sistema de likes y dislikes. En este todas las respuestas del chatbot tendran los botones de like y dislike que al presionarlos sacara un modal el cual le preguntara opcionalmente al usuario si quiere explicar su motivo. Despues de terminar esto, la pregunta, la respuesta, el comentario y si fue like o dislike se mandan al backend para crear un archivo de texto.

Estos se van a ir recolectando en una carpeta y luego un script tomara todos los archivos de texto y lo analizar he intentara brindar algunas ideas de lo que esta bien, lo que esta mal, que podria ser mejor, que podria ser peor y asi.



Implementacion

Historial

How do i remove and install the front panel in my ...

Nuevo chat

Configuración

Manually

Configuración

- Tipo de Respuesta (RAG)
 - Solo RAG**
Basado únicamente en el RAG. No se permite ampliar fuera de los manuales proporcionados.
 - RAG + Extra**
Basado en RAG pero amplía sobre puntos importantes o necesarios para el técnico.
 - RAG como Referencia**
Usa el conocimiento del modelo como base, consultando el RAG solo si es adecuado.
- Tono de Voz
 - Casual**
Lenguaje relajado y fácil de entender.
 - Normal**
El equilibrio estándar del modelo.
 - Formal**
Lenguaje técnico, preciso y

Resultados

TessTrain

	CER	Accuracy
Base	8.81%	91.19%
Pam2	7.23%	92.77%

E1.jpg

Mejora Neta: 17.9%

	CER	Accuracy
Base	65.43%	34.57%
Pam2	64.54%	35.46%

E2.jpg

Mejora Neta: 1.4%

Resultados

ColPali

Modelo	NDCG@5	MAP@5	MRR	Recall@10
MPNET	0.41	0.38	0.40	0.46
ColPali	0.67	0.61	0.65	0.757

Modelo	Tiempo de indexado en Segundos
MPNET	23.76s
ColPali	2,570s

Resultados

Fortalezas

- **Mejor comprensión de diagramas, tablas, layouts o documentos complejos.**
- **No requiere preprocesamiento de texto como limpieza de caracteres o manejo de tablas.**
- **No depende de texto extraíble en PDFs ni OCR.**

Limitaciones

- **Mayor costo computacional.**
- **Mayor latencia de Inferencia.**
- **Depende de la calidad de imagen.**

Conclusiones

- Se implementó el modelo ColSmol basado en la arquitectura de ColPali para generar embeddings basados en imágenes, recuperando exitosamente documentos en base a un query que requería información visual.
- Se implementó Tesseract junto con TesserTrain para entrenar un modelo para leer las etiquetas técnicas de imágenes y reconocer detalles del equipo con el que está trabajando el técnico.
- Se brindó una forma detallada de darles retroalimentación a los desarrolladores por medio de informes de varias respuestas

Recomendaciones

- Implementar un modelo de ColPali con más parametros.
- Hacer fine tuning del modelo con diagramas u otros elementos visuales del corpus para mejorar la inferencia.
- Ver formas mas avanzadas de dar recomendaciones

Bibliografia

Prieto, R. P. (2025, August 4). Automotive Document Intelligence with MongoDB Atlas Search. MongoDB. <https://www.mongodb.com/company/blog/innovation/automotive-document-intelligence-mongodb-atlas-search>

Bluon. (2024). Bluon AI MasterMechanic - HVAC AI. Bluon.ai. <https://www.bluon.ai/about>

Taylor, M. (2023, October 5). ChatGPT-Style AI Tool for HVAC Takes Aim at Labor Shortage. Achrnews.com; ACHR News. <https://www.achrnews.com/articles/153649-chatgpt-style-ai-tool-for-hvac-takes-aim-at-labor-shortage>

Faysse, M., Sibille, H., Wu, T., Omrani, B., Viaud, G., Hudelot, C., & Colombo, P. (2024). ColPali: Efficient Document Retrieval with Vision Language Models. ArXiv.org. <https://arxiv.org/abs/2407.01449>

Xenos, P. (2025, October 14). The hidden cost of missing documentation. Fieldcode. <https://fieldcode.com/en/resources/blog/hidden-cost-of-missing-documentation>

Muffat, A.-S. (17 de Abril de 2023). How Tech Doc Fails Service Technicians: A Webinar Recap. Obtenido de Fluid Topics: <https://www.fluidtopics.com/blog/field-service/how-tech-doc-fails-service-technicians/>



**Gracias por su
Atencion**

